Aligarh Muslim University

Secure Test Portal — AI Webcam Proctoring & Neighbor Detection

Module: Computer Vision Anti-Cheat	**Platform:** Electron.js / Node.js
Deployment: 100% Offline (Local LAN)	**Vision Engine:** Face-API / TensorFlow.js WebGL

1. Executive Summary & Purpose

In university computer laboratory coding examinations, visual collusion—specifically glancing sideways at an adjacent candidate's monitor or conferring with neighboring students—presents a persistent academic integrity vulnerability. The **AI Webcam Proctoring & Neighbor Detection** feature delivers continuous, on-device facial landmark and head-pose telemetry directly within the AMU Student Portal. It automatically flags neighbor-peeking behavior, issues localized student warnings, captures photographic evidence, and alerts invigilators over the local network in real time.

2. Key Highlights

100% Offline On-Device Computer Vision (No Cloud Required)

Operates completely without external internet connectivity. Neural network weights for TinyFaceDetector and FaceLandmark68 are stored locally in the application package, ensuring zero cloud dependency, zero external API latency, and maximum privacy compliance for air-gapped lab testing.

Biometric Facial Symmetry & Head Yaw Tracking

Extracts 68 facial landmark coordinates 7–8 times per second. By computing Euclidean symmetry ratios between the nose tip and outer eye corners, the algorithm accurately determines whether a student is facing their monitor, looking left, or looking right towards a peer.

Multi-Stage Violation & Grace Period Lifecycle

Includes intelligent temporal debouncing: brief natural blinks or minor neck movements (< 1.0s) are ignored; sustained head turns (≥ 1.0s) trigger an amber warning state; turns exceeding 2.0 seconds record an official violation, capture an evidence JPEG frame, and notify the teacher.

Floating Windows XP Picture-in-Picture (PiP) Interface

Features an authentic Windows XP-styled draggable, minimizable floating webcam window. Displays the live video feed (mirrored naturally) alongside an overlay canvas rendering real-time face bounding boxes and direction tags (✓ SCREEN FOCUS, ■ LOOKING RIGHT, ■ LOOKING LEFT).

Automatic Evidentiary Snapshot & Server Synchronization

When an infraction is verified, an offscreen canvas captures a compressed JPEG frame and immediately dispatches it via Socket.io to the Teacher Admin Server with candidate identity, seat number, and violation metrics.

3. Technical Details: How It Works

The proctoring system operates in an automated pipeline: **Frame Ingestion** → **Neural Landmark Inference** → **Mathematical Yaw Calculation** → **Temporal Thresholding** → **Alert Dispatch**.

■ 1. Frame Ingestion Captured via getUserMedia at 320×240 (15–20 FPS). Camera permissions are granted silently in Electron via session handlers without prompting user dialogs.	**■ 2. Landmark Detection** WebGL-accelerated TinyFaceDetector identifies face bounds; FaceLandmark68Net maps 68 anatomical coordinates across jawline, eyes, nose, and mouth.
■ 3. Geometric Yaw Analysis Computes nose tip (Point 30) displacement relative to eye corners (Points 36 & 45) and checks Euclidean symmetry ratio.	**■ 4. Alert & Evidence Dispatch** Upon reaching 2.0s threshold, captures JPEG snapshot, triggers student warning banner, and emits socket event to teacher dashboard.

Mathematical Formulation & Decision Boundaries

```
Symmetry Ratio = Distance(Nose Tip [30], Eye Right Outer [36]) / Distance(Eye Left Outer [45], Nose Tip [30])
Nose Offset X = (Nose Tip X - Eye Midpoint X) / Eye Span Width
```

- **Facing Screen (Normal):** Symmetry Ratio between 0.60 and 1.65; |Nose Offset X| ≤ 0.16. Status: *Green (Safe)*.
- **Looking Right (Neighbor on Right):** Symmetry Ratio > 1.65 or Nose Offset X > +0.16. Sustained ≥ 2.0s triggers *Violation*.
- **Looking Left (Neighbor on Left):** Symmetry Ratio < 0.60 or Nose Offset X < -0.16. Sustained ≥ 2.0s triggers *Violation*.
- **No Face Detected / Candidate Absent:** Zero faces detected for > 3.0s continuously triggers absence alarm.

4. Architecture & File Implementation Summary

The following table details all architectural files created or updated to establish the offline webcam proctoring pipeline across the student portal and admin server:

Component	File Path	Description
Detection Engine	`src/renderer/js/webcamProctor.js`	Core AI detection loop, head pose angle calculation, snapshot capture, and warning trigger.
PiP Styles	`src/renderer/css/webcam.css`	Windows XP styled draggable PiP box, status indicators, and warning animations.
Offline Models	`src/renderer/assets/models/`	Bundled model weights (tiny_face_detector and face_landmark_68).
Offline Library	`src/renderer/js/lib/face-api.js`	Standalone Face-API bundle with embedded TensorFlow.js WebGL runtime.
UI Integration	`src/renderer/index.html`	Added PiP widget HTML markup, CSS stylesheet link, and script references.
App Lifecycle	`src/renderer/js/app.js`	Hooks WebcamProctor.init() on startup, start() during exam, and stop() on submit.
Bridge API	`src/main/preload.js`	Exposed reportWebcamViolation safely to renderer via contextBridge.
Main Process	`src/main/main.js`	Granted media permissions automatically and forwarded violations via Socket.io.
Teacher Server	`AMUAdminServer/server.js`	Added webcam_violation socket listener to log alerts and forward evidence snapshots to the invigilator dashboard.